

Graphical Layer Abstraction Module



Gaming the Graphical Layer
(to Create Games and Other Stuff)

SAIFTYNET



London



Perl & Raku



Workshop



2026



Introduction :-\$ whoami

I am a full-time Orthopaedic Surgeon

Intermittent Hobbyist Programmer and Muck-arounder

Most of my coding is on the Terminal

Even the Graphics have been mainly Terminal Graphics

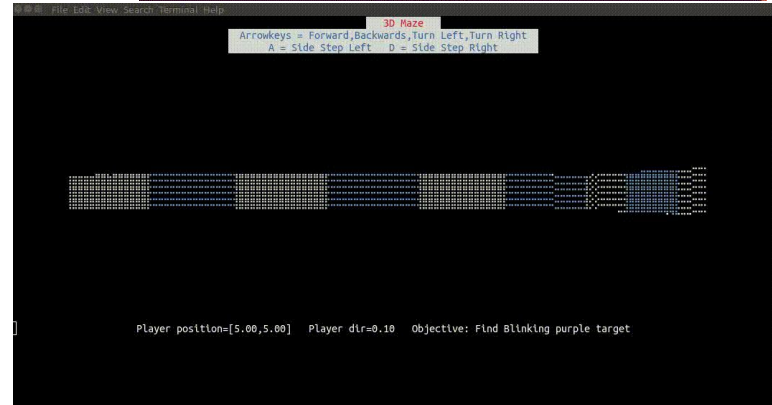
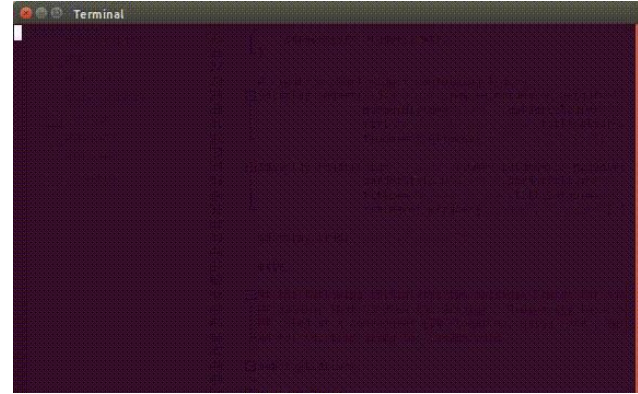
I wrote Term::Graille

I wrote a GUI Abstraction Tool GUIDeFATE (Graphical User Interface Development From A Text Editor)

Graphics Coding is Hard (for me)

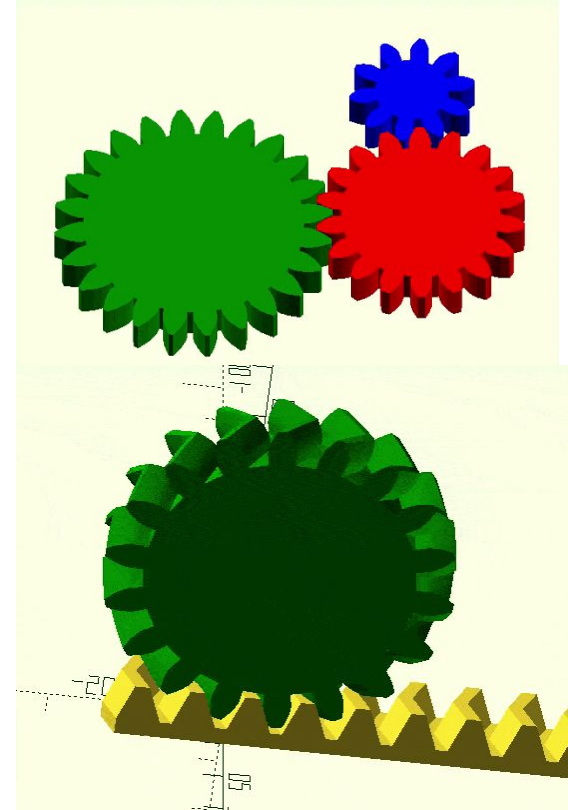
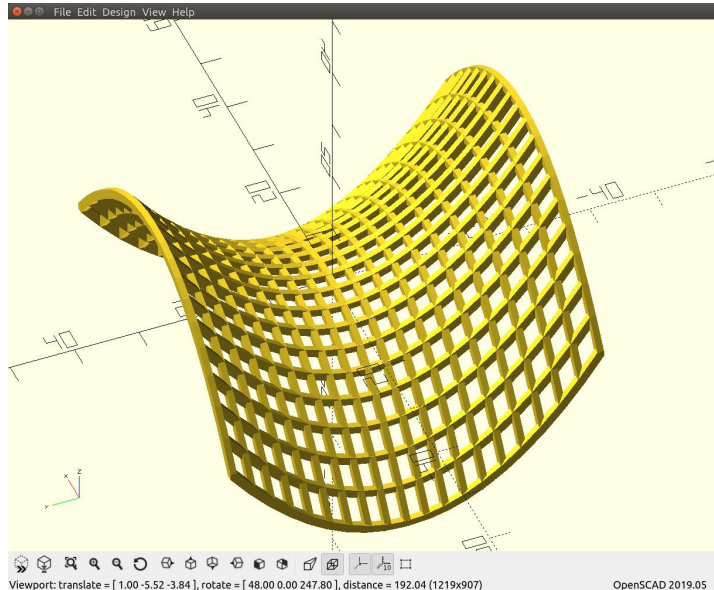
I tend to use Terminal ASCII Graphics

Term::Graille, Braille characters using
Perl's strength at string manipulation



Graphics Coding is Hard (for me)

Use Perl to generate scripts for another tool to create the graphics and animations.



OpenGL / SDL and other modules exist

- Easy for people who use it regularly
- Graphical games have been developed e.g. Frozen Bubble
- Powerful graphics tools possible e.g. Slic3r
- Enthusiastic Perl Graphical developers exist, Sanko Robinson [SANKO], Chris Marshall [ChM], Bob "grafman" Free etc

NOT easy for people who mainly code on servers and terminals, or like me just as hobby

Each module offers hundreds of different methods, constants

Each has a slightly different **workflow**

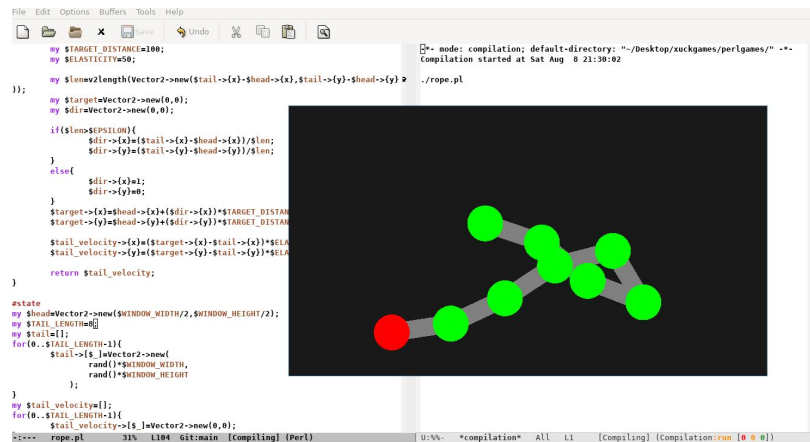
Each example provided blends Backend code with Game/Application Logic

Trigger - A Reddit Post

Phan Thanh Duy (u/First_Ad8230 on Reddit, Foolish4 on Github) posted a demo of interactive animated Rope, ported into Perl from another language. Used GLFW and OpenGL

Sanko Robinson replied with an SDL3 port of the same demo *within about 2 hours* from the initial post.

This is EASY for these guys



Parallel Learning - The Rosetta Stone of Graphical Layers

Create objects

Defined algorithms that manipulated the objects

Objects Interact with each other and with the user

Graphics primitives (triangles->lines & circles)

Collected User inputs (“Esc”, “q”, mouse)

Drew the graphics in a window

Update Render and Loop

“Game Logic”

Common for both

“Backend Logic”

Different BUT similar graphics primitives used in these demos

Abstraction

- Separate Game Logic from Inputs and Graphical Interface
- The Graphical Output and Keyboard/Mouse Inputs could be converted to a very simple API with consistent yields regardless of back-end
 - GLAM->key(<keyName>) # state of key
 - GLAM->mousePosition() # position of mouse
 - GLAM->button(<btnName>) # state of mousebutton
 - GLAM->{canvas} # the canvas object passed to the Game Object
 - GLAM->{canvas}->triangle() # a triangle primitive (all others are derived from this)
 - GLAM->{canvas}->thickLine() # a line of specified width (two triangles)
 - GLAM->{canvas}->circle() # a circle (multiple triangles)
 - GLAM->mainloop(&update) # the main loop

How-To Install the Backends

- Not trivial
- SANKO produced his SDL3 version because he couldn't install GLFW on his Windows Machine
- SDL3 requires Perl version 5.40 (most system Perls on Linux are lower)

For My Setup (Ubuntu 20.04 Perl v 5.30 ... yeah old, but ain't broke yet)

OpenGL

```
sudo apt install libopengl-perl  
sudo apt install libglfw3 libglfw3-dev  
sudo cpanm OpenGL OpenGL::GLFW
```

SDL3

```
sudo apt install perlbrew ; perlbrew install perl-5.40+  
Get dependencies from SDL3 Linux wiki (some fail)  
Manual install libssl  
sudo apt install libpipewire-0.2-1 libjack0 libjack-dev  
libpipewire-0.2-dev libpulse-dev libsndio7.0  
libsndio-dev  
cpanm SDL3
```

How-To

Install GLAM

- Code is on Github <https://github.com/saiftnet/GLAM>
- Place the GLAM folder and its contents into a path in @INC

use GLAM::OpenGL; # or GLAM::SDL

- Not yet ready for CPAN
- May need a GLAM::Any module



Built-In Provides

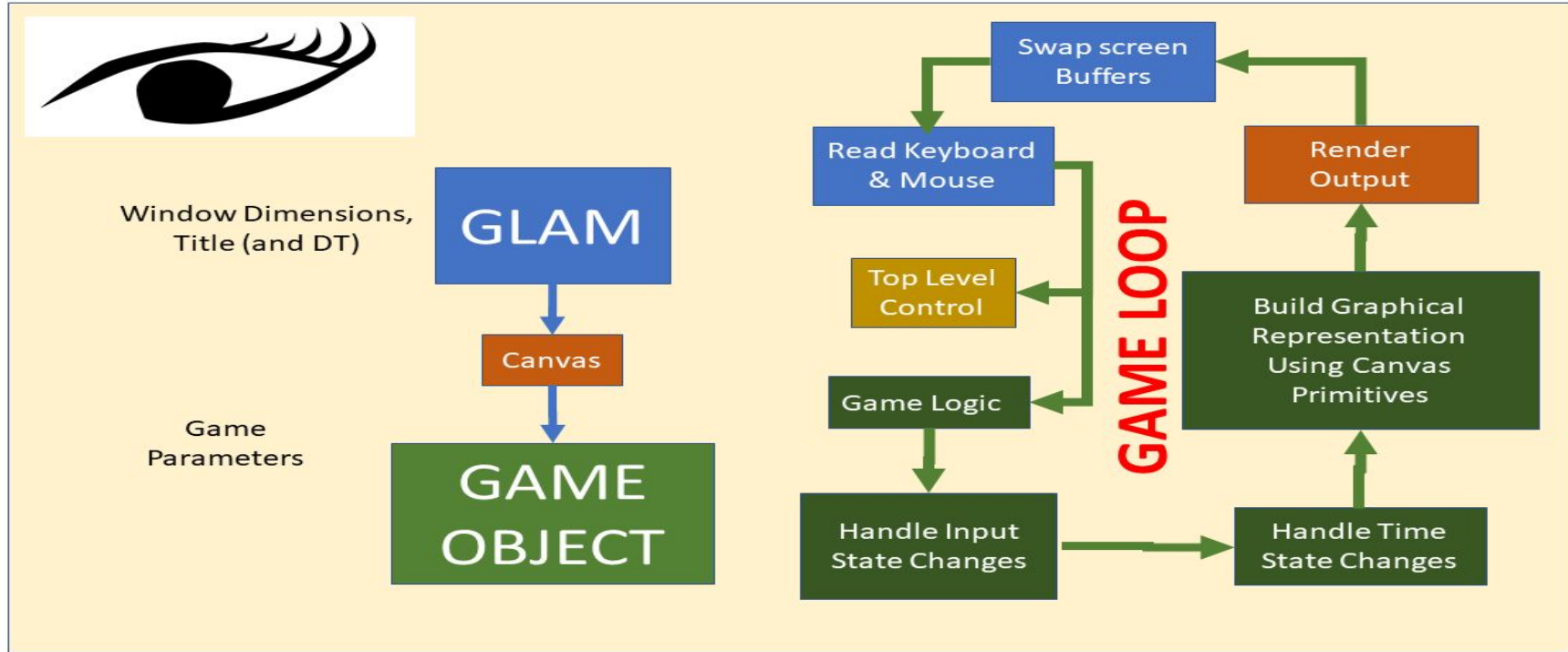
The GLAM Object methods `mousePosition`, `key`, `button`, `mainloop`

The Canvas Object methods `colour`, `triangle`, (`thickline`, `circle`, `quad` derived from `triangle`)

The Vector2 Object (2d Vectors and methods include `add`, `diff`, `mul`, `div`, `length`, `dot` product `cross` product, and `transforms`) unifies Game and Canvas vectors

(**The Ball Object** a circle object with radius, position, velocity, static and dynamic friction, collision detection, and bouncing) used in the Carrom demo.

The Architecture of a GLAM Application



The Game Objects

```
use GLAM::OpenGL; # change to `use GLAM::SDL` to use SDL3;

# Initialise the Graphics Layer (and create Canvas object)
my $gl = new GLAM({height=>400,width=>800,dt=>1/60,title=>"My Game"});
# Initialise the Game Object which uses the Canvas, and optional Game Parameters
my $gameObject = new GameObject( $gl->{canvas}, $gameParams);
# The mainloop needs to loop through updating the Graphics layer and Game Date
$gl->mainLoop(\&update);

sub update{
    my ($app)=@_ ;
    if($app->key("esc") || $app->key("q")){# keys to exit GameLoop
        print("goodbye!\n");
        exit(0);
    }
    $gameObject->update($app->{dt},<List_of_IO_etc>);
}

package GameObject;
```

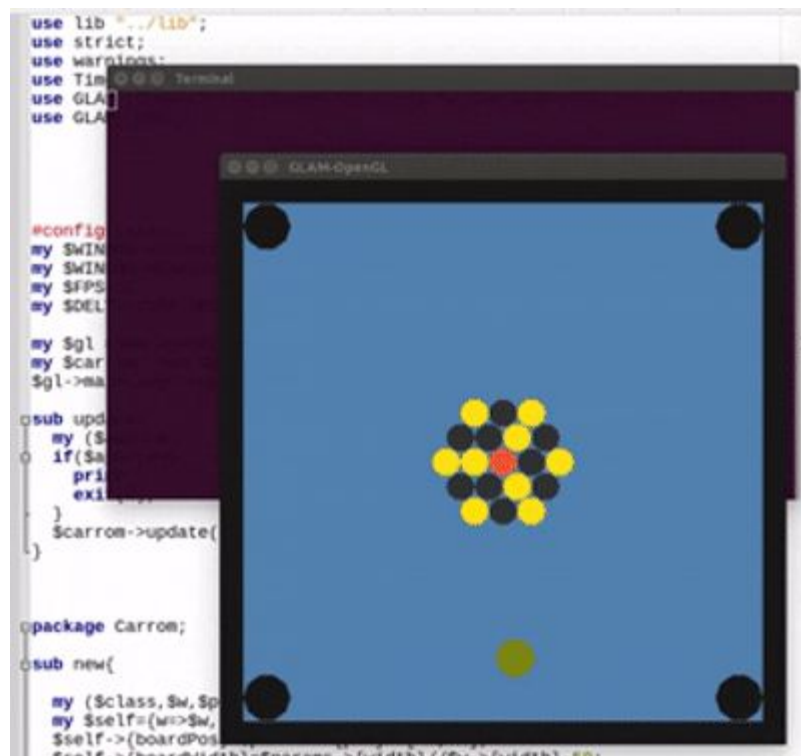

Summary In use (as a Graphics Layer newbie)

GLAM Provides

- Triangles to create all graphical elements
- Simple tests for mouse and keyboard states
- Simple Input capture, Game object update, Graphical Output Rendering Loop

The Game Object does everything else...

Examples



Conclusion

- Because of other more clever than I, I can now start more elaborate interactive graphical tools
 - Parallel demonstrations using the same frontend logic, but different backends allows easy isolation of the backend code
 - The backend code once isolated, can be abstracted.
-
- It would have been ***impossible*** for an old guy like me to learn to use SDL or OpenGL otherwise.

Acknowledgements

Phan Thanh Duy

Sanko Robinson

Ed J

Chris Marshall

London Perl Workshop